Week 2: input, output and files; booleans, functions, scope and modules

MIS41110 Programming for Analytics · 2026/27 · Week 2 lecture notes

These notes cover week 2's lecture, for revision. Read them beside its slides: every part names the pages it covers. They add the explanation behind each slide, the runs written out line by line, the mistakes the code invites with the message Python prints, the polls with their answers, and a few questions to check yourself, answered at the end. Common questions are answered where they come up. Others have written answers on the Q&A page, and the Function reference lists what each tool takes and gives back; both are on Brightspace, under Feedback. If you already program, the boxes headed **Good practice** are for you: the habits experienced programmers keep, next to the code they are about. The notes replace neither the lecture nor the practical: the sheet is where you practice.

From this week the code comes with the slides: `cafe_w2.py`, the café exactly as it stands at the end of the lecture, `register_sayings.py`, the second file it imports, `menu.txt`, the little file the reading demos open, and `w02_argv_start.py`. Put all four in one folder, and open your terminal in that folder: a file in another folder, or a terminal opened somewhere else, is the first thing I check when a program runs on my screen and not on yours. Run the files, break them, change them. If you break one beyond repair, download it again.

One habit for the first weeks: switch off your editor's AI suggestions, the gray text that offers to finish your line. Type the code yourself, read the error yourself, fix it yourself. In the exam nothing will suggest anything.

Where we started

Slides, pages 3 to 7.

Poll. One word: where does a program keep something so that it survives after the program ends? Anywhere outside the program's memory: a file, a folder, the disk, a database. Try it: type `pi = 3.14` in the Python shell, close it, open it again and type `pi`. Python has no idea. Everything a program holds in its variables dies with it. The plainest of those places, a file, is the one section 3 builds.

Recap. Before that, a debt from week 1: the float wall. The week ended on `3 lattes: 9.600000000000001`, three lattes at 3.20 that came to 9.6 with a tail of nonsense digits. The value was fine; the display was the problem. A question that tests it: state the output of `print(0.1 + 0.1 + 0.1)`. Here is why the tail appears, in three steps.

Step 1: a tenth never ends in binary. You know one third has no exact decimal: 0.3333333333 and on, forever. You cannot store forever, not in a computer and not in your head either, so you cut, write 0.33, and carry a small error. In binary, one tenth is that same kind of number: the pattern 1001 repeats forever. A float is 64 bits: 1 for the sign, 11 to place the point, 52 for the digits, and the leading 1 comes free, so 53 digits are kept and the rest is dropped. Dropping is rounding, the way two thirds to two decimals is 0.67 and not 0.66: the dropped tail is worth more than half of the last kept bit, so that bit goes up, and the stored tenth is a hair above one tenth.

Drawn, step 1 has four rows. A third in base 10 never ends and is cut. A tenth in base 2 never ends either: 53 bits are kept, boxed, and the tail beyond them is rounded away. That tail is worth more

than half of the last kept bit, so the last group, `1001`, becomes `1010`, and that is what is stored. And the 64 bits that hold a float are split 1, 11 and 52, which is why the box holds 53 bits and no more.

`1/3` in base 10: `0.3333333333...` never ends: cut, a tiny error

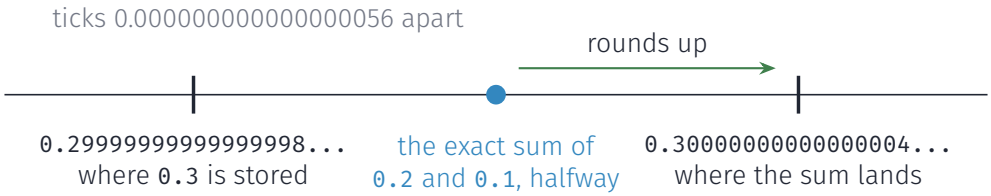
`1/10` in base 2: `0.000` `1 1001 1001 1001 ... 1001 1001` `1001 1001 ...`
53 bits kept rounded away

stored: `0.000` `1 1001 1001 1001 ... 1001 1010` rounded up



Step 2: the sum lands between two ticks. Under a microscope, the values a float can hold are ticks on a ruler, and near 0.3 they sit about 0.00000000000000056 apart. The exact sum of 0.2 and 0.1 falls exactly halfway between two ticks, and the rounding rule sends it up to the next one.

On the ruler, the left tick is where `0.3` itself is stored, at 0.2999999999999998..., a hair below it. The exact sum falls halfway to the next tick and rounds up, so the sum and `0.3` land on two different ticks, and that is why they print differently.



Step 3: Python prints the shortest decimal that names a tick. That is why `0.1` still prints as `0.1`, and why the tick the sum lands on prints with its long name. Python's float is the same 64 bits on every machine you will meet, so you and I see the same digits.

```
>>> 0.2 + 0.1
0.30000000000000004
>>> 0.1 + 0.1
0.2
>>> 0.1 + 0.1 + 0.1
0.30000000000000004
```

The last sum is the recap's question. The answer is exactly what Python prints: `0.3`, then fifteen zeros, then a 4, never `0.3` alone.

Why does `0.1 + 0.1` come out clean? Adding a number to itself doubles it, and doubling in binary only moves the point: the same 53 bits, nothing dropped, nothing to round. The two rows below hold the same bits; only the power of two after them changes, by one.

`0.1` `1 1001 1001 ... 1001 1010` $\times 2^{-4}$

`0.1 + 0.1` `1 1001 1001 ... 1001 1010` $\times 2^{-3}$

the same 53 bits, the point moved by one: exactly the tick `0.2` names

The error is not always there; it shows when a rounding lands off the tick you expect. `0.1 * 2` is `0.2` and `0.1 * 4` is `0.4`, but `0.1 * 3` is `0.30000000000000004`, while ten thirds added three times

is a clean `10.0`. You cannot see it coming, which is the whole lesson: never depend on the last digits. The bits and the ticks are for the curious. The sentence to keep for the exam: **a tenth has no exact binary form, so floats carry tiny errors that can show when you add or multiply.**

Week 1's follow-up is on the Q&A page, under Week 1.

1 · Output: talking to people

Slides, pages 8 to 13.

Programs eat data and produce data

Page 8. Step back and look at what a program is: a machine for data, like a function in mathematics. `x` goes in, `f` does its work, `y` comes out. The data comes in from the keyboard, from files, or from program arguments, and goes out to the screen, to files, to a printer, to an action. And the pieces chain: keyboard into a file, the file onto the screen, one program's output into another. Last week the café had no input at all, since every quantity was typed into the code, and one output, the screen. By the end of the week it reads the keyboard and writes files.

Program arguments look new, and you have used them already: run a single line of Python straight from the terminal, and `python3` is the program, `-c` and the code after it are its arguments:

```
$ python3 -c "print('Hello')"
Hello
```

Section 4 gives your own programs arguments.

Three ways to print data, and the fix

Page 9. Here the debt is paid. Watch the same latte total printed three ways, and watch the tail appear every time:

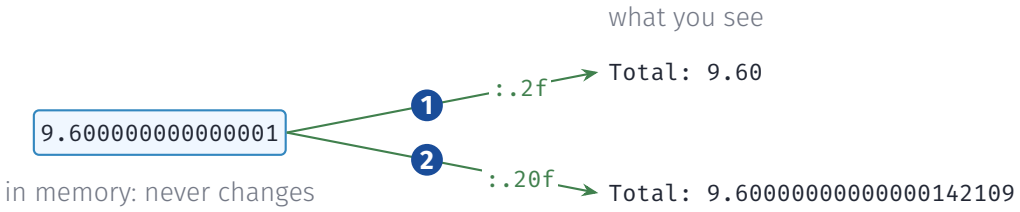
```
>>> latte_price = 3.20
>>> lattes = 3
>>> print("Total:", latte_price * lattes)          # commas: mixed types, auto-spaces
Total: 9.600000000000001
>>> print("Total: " + str(latte_price * lattes))   # convert + concatenate (Week 1)
Total: 9.600000000000001
>>> print(f"Total: {latte_price * lattes}")        # f-strings: expressions in braces
Total: 9.600000000000001
```

Three styles, three different jobs for you. With `+` you build the whole string yourself, so every number has to go through `str()` first. With commas you hand `print` several arguments and it converts each one and puts a space between them. With an f-string you write the sentence the way you want to see it, and drop any expression into the braces: what you write is what you get, which is why it is the style I use most. All three print the tail, because the value in memory really is off, for the reason above. Now the fix:

```
>>> print(f"Total: {latte_price * lattes:.2f}")
Total: 9.60
>>> print(f"Total: {latte_price * lattes:.20f}")
Total: 9.60000000000000142109
```

After the colon inside the braces, `.2f` means "format as a float with two digits after the point". The wall is dead. We did not change the value; we display money the way a register does, to the cent. Widen the spec to twenty digits and you see the tail we just rounded away.

One value sits in memory, and it never changes. ❶ `:.2f` shows it to the cent. ❷ `:.20f` shows twenty digits of the same value, tail and all. The spec decides what you see; the value stays what it was.



Poll. What does `print("total:", 7, 2)` print: `total: 7 2`, `total:72`, `total: 7, 2`, or an error? It prints `total: 7 2`. `print` turns each of its three arguments into text and puts one space between neighbors. It adds no commas of its own and glues nothing together, and mixed types are fine here, unlike with `+`. One space per comma is also why a receipt lines up its columns with format specs, not commas.

The comma style has a setting you can change. `help(...)` shows the documentation of any function, and `help(print)` lists `sep`, the separator it puts between arguments, a space unless you say otherwise:

```
>>> print("latte", "espresso", "muffin", sep=" | ")
latte | espresso | muffin
```

f-strings

Page 10. Put an `f` before the opening quote, and inside any pair of braces Python evaluates whatever you write: a variable, a sum, a call.

```
>>> students = 70
>>> lecturers = 1
>>> place = "room"
>>> print(f"There are {students + lecturers} people in this {place} now.")
There are 71 people in this room now.
>>> current_year = 2026
>>> birth_year = 2000
>>> print(f"You turn {current_year - birth_year} this year")
You turn 26 this year
```

The types inside the braces still have to agree with each other, a year minus a year, never a year minus a name, but the f-string turns the answer into text for you. And the rule to file away for the drill on page 13: **the expression is evaluated the moment the string is built**, not when it is printed.

Good practice. f-strings are how modern Python formats text, and they carry a debugging trick worth knowing, one the slides do not cover: a `=` after the expression prints the expression and its value together.

```
>>> total = 17.0
>>> print(f"{total=}")
total=17.0
>>> print(f"{total * 1.1=:.2f}")
total * 1.1=18.70
```

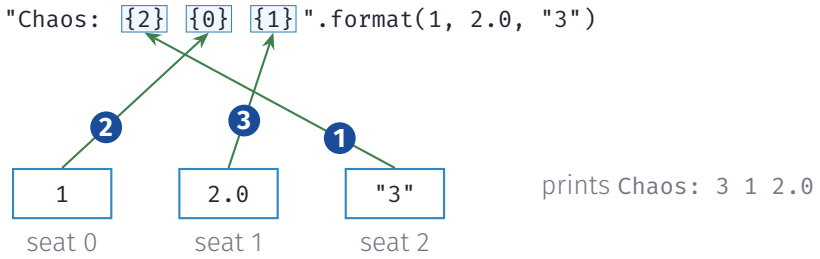
`str.format()`: the older sibling

Page 11. Before f-strings there was `.format`, and older code and exam questions still use it, so you need to read it even if you rarely write it. The holes are empty or numbered braces, and the values come in the call at the end:

```
>>> print("There are {} people in this {} now.".format(71, "room"))
There are 71 people in this room now.
>>> print("Order: {0} {1} {2}".format(1, 2.0, "3"))     # by position
Order: 1 2.0 3
>>> print("Chaos: {2} {0} {1}".format(1, 2.0, "3"))     # any order (count from 0!)
Chaos: 3 1 2.0
>>> print("{:d} = {:f} = {:e}".format(3, 3, 3))         # force a type
3 = 3.000000 = 3.000000e+00
```

Think of the values in the call as a row of seats, numbered from 0, the way programming counts. Python is in good company there; if you learned MATLAB, which counts from 1, I am sorry. `{2}` takes the value in seat 2, the third one, so the chaos line pulls the third, then the first, then the second, and a number can be used twice.

Each numbered hole takes the value in the seat with its number. ❶ `{2}` takes seat 2, the `"3"`. ❷ `{0}` takes seat 0, the 1. ❸ `{1}` takes seat 1, the 2.0. The holes fill in the order the template lists them, whatever order the seats are in, which is why the line prints `Chaos: 3 1 2.0`.



The type codes work in f-strings too: `d` for an integer, `f` for a float, `e` for scientific notation, `s` for a string. Give `d` a float and Python refuses rather than convert it:

```
>>> print(f"{9.6:d}")
ValueError: Unknown format code 'd' for object of type 'float'
```

That is what you learn by trying things: `d` wants a whole number and does no rounding for you.

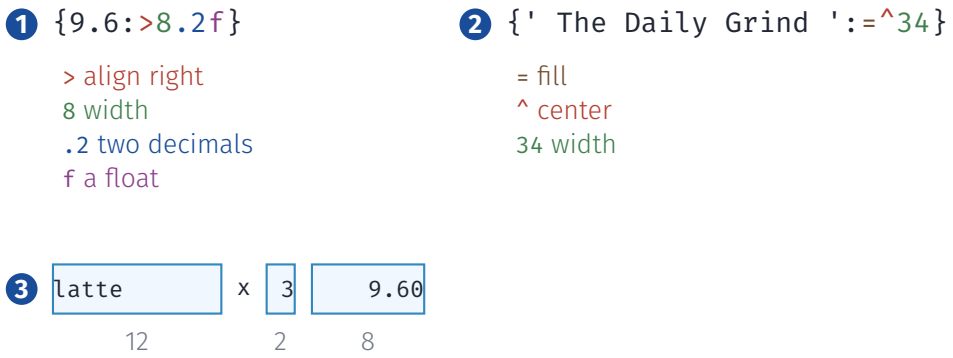
Format specs earn their keep: the receipt table

Page 12. A spec can set the width, the precision, the alignment and a fill character, which is exactly what a receipt needs:

```
>>> print(f"{' The Daily Grind ':=^34}")
======== The Daily Grind =========
>>> print(f"{'latte':<12} x {3:>2} {9.6:>8.2f}")
latte        x  3     9.60
```

`:<12` left-aligns in 12 characters, `:>2` right-aligns in 2, `:>8.2f` right-aligns in a field of 8 with two decimals, and `:=^34` centers in 34 characters and fills the gaps with `=`. The fill character comes first, then the alignment: `<` left, `>` right, `^` center. When the gap does not split evenly, the extra character goes on the right, which is why the header has eight `=` before the name and nine after.

Read a spec from the colon on; each part keeps its color in both. (1) In `{9.6:>8.2f}`, `>` aligns right, `8` is the width, `.2` the decimals and `f` the type. (2) In `{' The Daily Grind ':=^34}`, the `=` before the alignment is the fill, `^` centers, and `34` is the width. (3) On the row, each field keeps its width whatever the value, and the text between the fields prints as written, so every row of the receipt lines up under the one before.



Build the shop's banner the slow way first, because the story is worth more than the spec. A few stars on each side, by hand. Then more stars, copied and pasted. The moment you catch yourself copy-pasting, a warning should flash in your head: in programming we try to be lazy, and copy-paste is the wrong kind of lazy. So `"*" * 5` instead of five stars typed. Then a variable for the count, so one edit changes both sides; then a variable for the symbol; then `len` to count the characters of the name instead of counting them on your fingers; and at last a computed width, with the symbol and the width passed into the spec itself:

```
>>> name = " The Daily Grind "
>>> print(len(name))
17
>>> symbol = "*"
>>> stars = 5
>>> print(f"{name:{symbol}^{len(name) + 2 * stars}}")
***** The Daily Grind *****
```

Change `stars` to 10 or `symbol` to `"="` and the banner follows. That is the habit I want from you: solve the problem naively, then ask how to stop repeating yourself, and break the problem into pieces until each one is a single line.

Why bother with widths at all, when a tab (`\t`) also lines things up? Because a tab jumps to the next tab stop, so a product with a longer name lands its price one stop further right and the column breaks. A width locks the column, whatever you put in it. One thing a width does not do is cut: a value longer than its field prints whole.

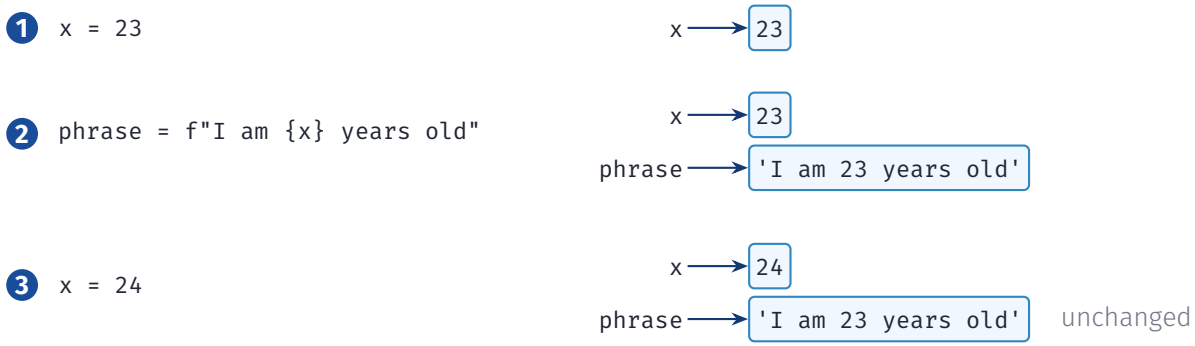
```
>>> print(len(f"{'The Daily Grind':^10}"))   # 15 characters asked into 10: nothing is cut
15
```

A heads-up for the self-study: the free books teach f-string basics, not these alignment specs, so for this part the slides and these notes are your reference.

Predict, then run

Poll. The drill's first line sets `x = 23`, builds `phrase = f"I am {x} years old"`, then sets `x = 24`. What does `print(phrase)` show: 23, 24, both, or an error? It shows `I am 23 years old`. An f-string is evaluated once, when it is built, so `phrase` holds finished text, made while `x` was 23. Rebinding `x` reaches nothing already built: week 1's rebinding rule, in a new hat. The 24 belongs to the second print, which builds a new string.

Follow the names. ❶ `x = 23` puts the name `x` on 23. ❷ The f-string is built now, with the 23, and `phrase` points at the finished text `'I am 23 years old'`. ❸ `x = 24` moves `x` alone; the text was finished before, so nothing in it changes.



Page 13. The whole drill, run:

```
>>> x = 23; phrase = f"I am {x} years old"; x = 24
>>> print(phrase); print(f"I am {x} years old")
I am 23 years old
I am 24 years old
>>> print("|{0:-^12}|".format("Center"))
|---Center---|
>>> print("|{0:{1}<12}|".format("Left", "-"))      # the fill char as an argument!
|Left--------|
>>> print("|{0:{1}>12}|".format("Right", "-"))
|-------Right|
>>> print("{s} = {n}".format(n=3, s="three"))      # named placeholders
three = 3
```

A semicolon puts several statements on one line, and Python still runs them in order, one after the other, like reading a story. In the centered line, count the characters: 14, not 12, because the 12 is the width of the field between the braces, and the two pipes are outside it. The last line names its holes, `s` and `n`, so the order of the values in the call no longer matters.

Common mistakes. Gluing a number with `+` and forgetting `str()`, and forgetting the `f`, which turns the braces into ordinary characters:

```
>>> print("Total: " + latte_price * lattes)
TypeError: can only concatenate str (not "float") to str
>>> print("Total: {latte_price * lattes:.2f}")       # no f in front
Total: {latte_price * lattes:.2f}
```

The second one is easy to miss when the text goes into a file rather than onto the screen. And `d` on a float, above. And two alignments in one spec: a spec holds one alignment and one width, so this stops before anything prints:

```
>>> print(f"{'latte' : >2 <10}")
ValueError: Invalid format specifier ' >2 <10' for object of type 'str'
```

To indent a column, write the spaces as text in front of the field; the bar shows where the field ends:

```
>>> print(f"    {'latte':<10}|")
    latte     |
```

Check yourself.

- **1.1** What does this print?

```
price = 2.50
label = f"espresso {price:.2f}"
price = 2.80
print(label)
```

- **1.2** What does `print(f"{'cocoa':*^8}")` print, character for character?
- **1.3** What does `print("{1}-{0}-{1}".format("tic", "tac"))` print?

See also. Sheet 2, exercise 4 (the supply table) and exercise 1 (b). The Function reference, Week 2, Talking and listening. py4e chapter 6, or Automate the Boring Stuff chapter 8, for strings.

2 · Input: listening

Slides, pages 14 to 16.

`input()`

Page 14. Now the café learns to listen. `input` does three things: it prints a prompt, it waits for the keyboard, and it hands back whatever was typed. Run `input()` with no prompt at all and the program just sits there with a blinking cursor. Is it frozen? No, it is waiting for you, and nothing says so. Always give it a prompt:

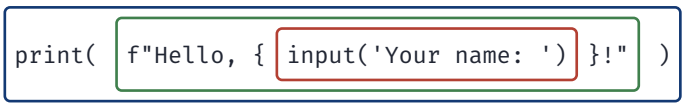
```
name = input("Your name: ")
print(f"Hello, {name}!")
```

Save the answer in a variable the moment you get it, or it is gone: nothing uses it, so the garbage collector from week 1 clears it away. You can also put `input` straight inside an f-string:

```
print(f"Hello, {input('Your name: ')}!")
```

Python then works from the innermost call outward: first it asks for the name, then it builds the string, then it prints. Legal, and harder to read, so use it sparingly. The prompt sits in single quotes because the f-string uses double ones: from Python 3.12 the same quotes work in both places, but an older Python stops at that line with `SyntaxError: f-string: unmatched '('`, so alternate them.

Each call sits inside the one that needs its answer. ❶ `input` runs first: it asks, you type Ada, and it hands back `'Ada'`. ❷ The f-string is built with that answer: `'Hello, Ada!'`. ❸ `print` runs last and shows it. A call waits until everything inside its parentheses has a value.



- ❶ `input` asks; you type Ada; it hands back `'Ada'`
- ❷ the f-string is built: `'Hello, Ada!'`
- ❸ `print` shows `Hello, Ada!`

The rule to burn in: **`input` hands back a string, always, without exception.** Type a number and you still get text:

```
>>> print(type(input("Give me a number: ")))
Give me a number: 22
<class 'str'>
```

The `input()` trap

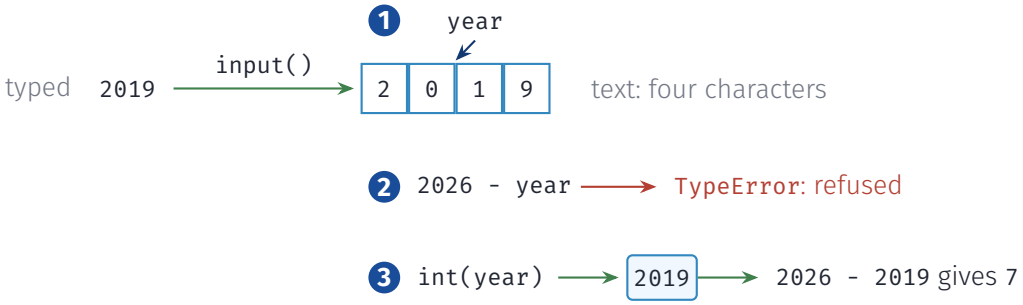
Page 15. Here is the trap, the one you will hit first:

```
>>> year = input("Which year did you first visit us? ")
Which year did you first visit us? 2019
>>> print(2026 - year, "years of loyalty!")
TypeError: unsupported operand type(s) for -: 'int' and 'str'
>>> print(2026 - int(year), "years of loyalty!")
7 years of loyalty!
```

Read the error, because the error is teaching you: it names both types. You tried to subtract text from a number; `input` gave you the characters 2, 0, 1, 9, not the number. The fix is week 1's conversion, and the habit that saves you every time is to convert the instant you read:

```
quantity = int(input("How many? "))
```

❶ `input` hands back what was typed as text, `'2019'`: four characters, not a number. ❷ Subtracting text from a number is refused, with the `TypeError` above. ❸ `int(year)` builds the number 2019 from the text, and the subtraction works: 7.



Good practice. Validate at the boundary. Convert and check what the user typed in one place, the moment it arrives, and let the rest of the program trust its values: a number that might still be text, three functions deep, is a bug waiting for its input. Week 5 gives that one place its safety net, `try` and `except ValueError`.

In the project. Your desk has a text interface that reads what its user types, and sensible error handling is one of the seven things the project's code is marked on (the Project rules page lists them).

The café takes its first real order

Page 16. The café takes its first real order. It asks for the cashier, greets them through the second file, then asks how many espressos, lattes and muffins, converting each answer the moment it arrives:

```
cashier = input("Cashier name: ")
print(register_sayings.welcome(cashier))

espresso_qty = int(input("How many espressos? "))
latte_qty = int(input("How many lattes? "))
muffin_qty = int(input("How many muffins? "))
```

Run it, type `Ada`, then `2`, `3` and `1`, and you get this:

```
$ python3 cafe_w2.py
Cashier name: Ada
Welcome to The Daily Grind, Ada!
How many espressos? 2
How many lattes? 3
How many muffins? 1
======== The Daily Grind =========
espresso     x  2     5.00
latte        x  3     9.60
muffin       x  1     2.40
==================================
TOTAL                17.00
That is 17 euro(s) and 0 cent(s).
Big order (over 10.00)?  True
Busy start (5+ items)?   True
Revenue on file: 17.00
Thank you, come again!
```

It listened, and the money is honest. The lines under the receipt are previews: section 3 earns the revenue line, section 5 the two questions, section 6 the euros and cents, and section 8 the welcome and the goodbye.

Now the part I want you to feel: one question per product is clumsy. Three products, three hard-coded lines; twenty products would be twenty lines, and there is still no way to say "keep asking until the customer is done". That itch has a name, a loop, and loops are next week.

Common mistakes. Doing arithmetic on what `input` returned, above. And typing anything that is not a whole number at a quantity prompt. The word `two` stops the café with

```
ValueError: invalid literal for int() with base 10: 'two'
```

and pressing Enter on an empty line gives the same error with nothing between the quotes. `int()` cannot read either, and nothing catches the error yet. That is week 5's story.

Check yourself.

- **2.1** `qty = input(...)`, and the user types `4`. What does `print(qty * 2)` show, and what would make it show `8`?
- **2.2** In `print(len(input("Name? ")))`, which call runs first, which runs last, and what prints when you type `Ada`?

See also. Sheet 2, exercise 2. The Function reference, Week 2: the `input(PROMPT)` row. py4e chapter 2.

3 · Files: remembering

Slides, pages 17 to 22.

Data lives in files

Page 17. The café can listen now, and everything it hears dies when the program ends. To remember overnight it needs files, and reading and writing a file works much like reading the keyboard and writing the screen. In fact C, the language Python itself is written in, treats those channels as three files: **stdin**, the keyboard, **stdout**, the screen, and **stderr**, the error channel. When Python shouts at you in your terminal, that is stderr.

The extension on a file name, `.txt`, `.csv`, `.py`, is a label for you, the human. Python runs a script whatever its name ends with. Where the extension does matter is `import`, section 8: it only finds files whose names end in `.py`. Underneath, every file is binary, and a text file is character codes, week 1's ASCII story, in an encoding such as UTF-8.

Opening a file

Page 18. `open` takes the file's name and a mode:

```
menu = open("menu.txt", "r")     # or just open("menu.txt"): "r" is the default
```

Mode	Meaning	after writing twice
`"r"`	**read** (default)	n/a
`"w"`	**write:** starts the file fresh (overwrites!)	`line 2` (the last only)
`"a"`	**append:** adds to the end	`line 1` + `line 2`
`"r+"`	read *and* write	not demoed

Reading is harmless: several programs can read the same file at once. Writing deserves respect: `"w"` empties the file the moment it opens it, before you write anything. `"a"` keeps what is there and adds at the end. Both create the file if it does not exist yet.

The mental model that saves you: the variable you get back is not the contents of the file. It is a **file object**, a pointer to the file on disk plus a bookmark of where you are reading. Print it and you see the object, not the menu:

```
>>> menu = open("menu.txt")
>>> print(menu)
<_io.TextIOWrapper name='menu.txt' mode='r' encoding='UTF-8'>
>>> menu.close()
```

(The encoding at the end can read differently on your machine: Python 3.15 prints `utf-8` everywhere, and an older Python on Windows often `cp1252`.) And close what you open, the way you close a library book when you are done so the next person can open it. You know the problem from USB keys: try to eject one while a document on it is still open, and the computer refuses, because something is using the file. Garbage-collecting the object never deletes the file itself.

`menu.txt`, posted with this week's files, holds four lines:

```
espresso, single shot 2.50
latte, with oat milk 3.20
blueberry muffin, warm 2.40
tea, pot for one 2.20
```

Good practice. Experienced Python code opens a file with `with`, which closes it at the end of the block even when an error interrupts the block, and names the encoding. Before Python 3.15 the default depends on the machine (the shell above printed UTF-8 on macOS; Windows often says otherwise); from 3.15 it is UTF-8 everywhere, and naming it keeps the file reading the same on every version:

```
with open("menu.txt", encoding="utf-8") as menu:
    first_line = menu.readline()
```

The course writes `open` and `close` by hand so you see both steps; the two do the same job.

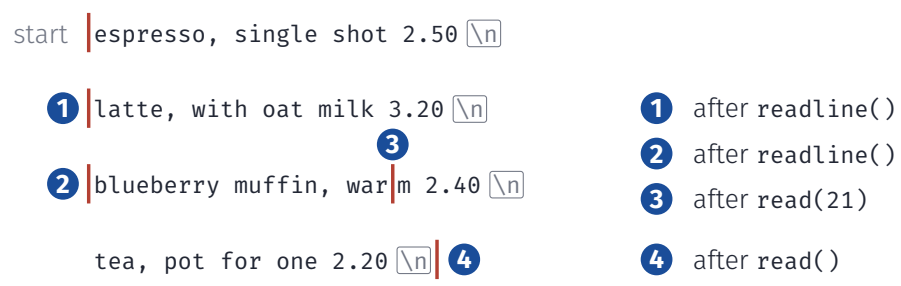
Reading: the bookmark moves

Page 19. Watch the bookmark move. In the shell, a line on its own shows the value it gives back, quotes and all, so you can see what you got:

```
>>> menu = open("menu.txt")
>>> menu.readline()
'espresso, single shot 2.50\n'
>>> menu.readline()                 # the object REMEMBERS where you are: line 2
'latte, with oat milk 3.20\n'
>>> menu.read(21)                   # exactly 21 characters
'blueberry muffin, war'
>>> menu.read()                     # everything from the bookmark to the end
'm 2.40\ntea, pot for one 2.20\n'
>>> menu.close()
>>> menu.readline()
ValueError: I/O operation on closed file.
```

The second `readline` gives the second line, because the object remembered where it stopped. `read(21)` gives exactly 21 characters, and `read()` everything that is left, which on a big file is the whole thing in memory at once. Once the file is closed, the object refuses to work. Look at the `\n` at the end of each line: every line read from a file brings its own line break, which is why `print(menu.readline())` leaves an empty line under the text, the file's break plus `print`'s own.

Here is the bookmark after each call, on the file's own four lines; `\n` is the line break each one ends with. It starts before the first character. ❶ `readline()` reads up to and including the first line break, so the bookmark lands at the start of line 2. ❷ The second `readline()` does the same with line 2. ❸ `read(21)` counts 21 characters and stops where they end, inside the word `warm`: a read by count does not care where a line ends. ❹ `read()` takes everything that is left and leaves the bookmark at the very end.



The bookmark, explicitly: `tell()` and `seek()`

Page 20. Think of the bookmark as the blinking cursor in a document, one you cannot see. `tell` says where it is, counted in bytes, and in plain ASCII text like this menu a byte is a character; an accented letter such as é takes two in UTF-8. `seek` moves it:

```
>>> book = open("menu.txt")
>>> book.tell()
0
>>> book.readline()
'espresso, single shot 2.50\n'
>>> book.tell()                     # 26 characters and the line break
27
>>> book.readline()
'latte, with oat milk 3.20\n'
>>> book.tell()
53
>>> book.seek(27)                   # back to the start of line 2
27
>>> book.readline()                 # so the same line comes back
'latte, with oat milk 3.20\n'
>>> book.close()
```

You rarely need this at the café, but it is examinable, and it explains why two `readline()` calls give two different lines: the bookmark moved.

Revenue that survives closing time

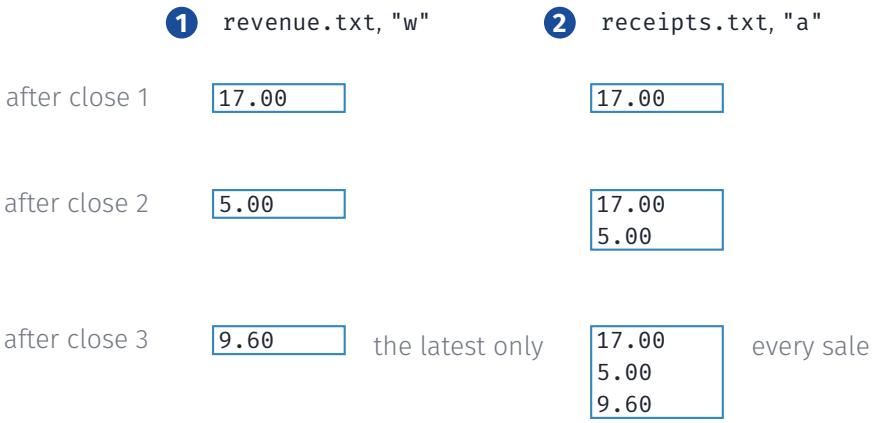
Page 21. Now the café remembers its money, with two files and two jobs:

```
revenue_file = open("revenue.txt", "w")       # "w": start the file fresh
revenue_file.write(f"{total:.2f}")            # write() wants a string -- f-strings deliver
revenue_file.close()

log = open("receipts.txt", "a")               # "a": append to the running log
log.write(f"{total:.2f}\n")
log.close()
```

`revenue.txt` is opened in `"w"`, because we want one figure, fresh at every close, so overwriting is exactly right. `receipts.txt` is opened in `"a"`, because the log should grow. And look at the names. A short name like `rev` makes sense now, with the slide in front of you; next week, reading your own code, you will have no idea what it means. `revenue_file` costs nine more characters and says what it is.

Here are both files over three closes, with totals of 17.00, 5.00 and 9.60. (1) `"w"` empties `revenue.txt` the moment it opens, so the file only ever holds the latest total. (2) `"a"` keeps what is there and adds at the end, so `receipts.txt` grows by one line per close. One figure fresh at every close, and a log that remembers every sale: the mode is the choice between the two.



`write` adds exactly what you give it and nothing else, not even a line break. Append twice without one and the two figures glue together:

```
>>> log = open("practice_log.txt", "a")
>>> log.write("2.20")
4
>>> log.write("3.20")
4
>>> log.close()
>>> check = open("practice_log.txt")
>>> check.read()
'2.203.20'
>>> check.close()
```

(The 4 is `write` telling you how many characters it wrote; a program ignores it.) That is why the café writes `f"{total:.2f}\n"` to its log: the `\n` puts each total on its own line. And the trap that catches everyone, `write` takes strings only:

```
>>> values = open("scratch.txt", "w")
>>> values.write(3.14)
TypeError: write() argument must be str, not float
>>> values.close()
```

Convert on the way out, just as you convert on the way in. One more warning: when you copy and paste an `open` line, check its mode. A pasted `"w"` where you meant `"r"` empties the file you wanted to read.

Proof it survived: write, quit, read back

Page 22. The café reads its revenue back at the end of every run:

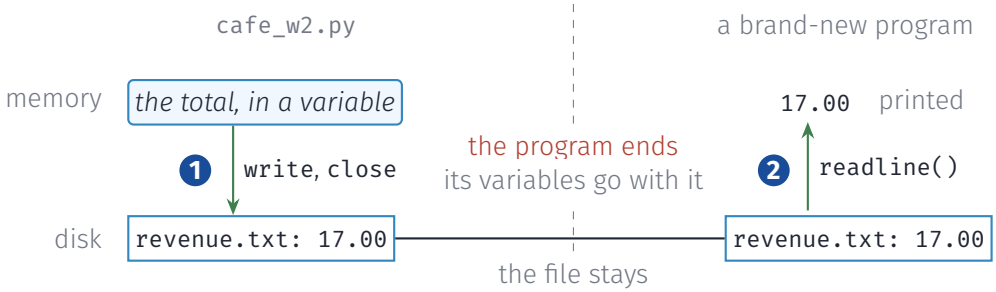
```
revenue_file = open("revenue.txt", "r")       # read it back: proof it survived
saved = revenue_file.readline()
revenue_file.close()
print(f"Revenue on file: {saved}")
```

That alone proves little, since the same program just wrote it. So do it the hard way: run the café, close the terminal entirely so every variable is gone, open a fresh terminal, and run a brand-new one-line program that knows nothing except how to read the file:

```
$ python3 -c "print(open('revenue.txt').readline())"
17.00
```

New program, no memory of the sale, and the money is still there. A variable dies with the program; a file outlives it.

Two programs, one file. ❶ The café writes its total to `revenue.txt` and ends, and every variable it held goes with it. ❷ A brand-new program, which never saw the sale, reads the file and prints `17.00`. The file is the only thing that crossed from one program to the next.



One caution for later: reading a file that does not exist yet, the very first morning before any revenue is saved, stops the program:

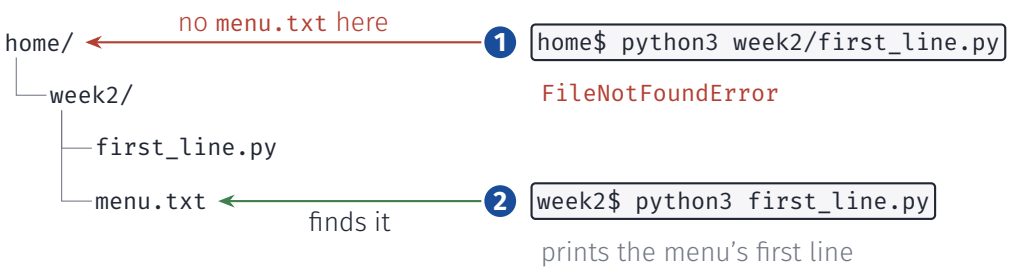
```
FileNotFoundError: [Errno 2] No such file or directory: 'revenue.txt'
```

Guarding that cleanly needs week 5's exceptions. Until then, read only files you just wrote.

> **In the project.** Saving your results and your leaderboard in plain text is another of the seven, and the brief asks for them to load back as well. This section's `"w"` against `"a"`, chosen on purpose, is the tool, and the write, quit, read-back proof is the test to repeat.

Common mistakes. Running from the wrong folder: `open("menu.txt")` looks in the folder your terminal is in, not the folder your code is in, so a terminal somewhere else gives `FileNotFoundError`. It is the first thing to check when a file will not open. Opening with `"w"` a file you meant to read. Writing a number without converting it. Forgetting `close()`: what you wrote may not reach the disk when you expect it. And reading after closing, which raises `ValueError`.

The first of those, drawn, with a one-line program in `week2/`, `first_line.py`, that runs `print(open("menu.txt").readline())`. ❶ From the home folder, Python finds and runs the program, but `open` looks in the home folder, finds no `menu.txt` there, and stops with `FileNotFoundError: [Errno 2] No such file or directory: 'menu.txt'`. ❷ From inside `week2/`, the terminal's folder is the code's folder, and the same `open` finds the file and prints its first line. A bare file name is looked up from where the terminal stands, never from where the code lives.



Check yourself.

- **3.1** `tips.txt` holds the line `1.50`. A program opens it with `"w"`, writes `"2.00\n"` and closes it, then opens it with `"a"`, writes `"0.75\n"` and closes it. What does the file hold now?
- **3.2** After `menu = open("menu.txt")`, why does `print(menu)` not show the menu, and what would?
- **3.3** With `menu.txt` above: after `menu = open("menu.txt")` and `menu.read(9)`, what does `menu.readline()` give back?

See also. Sheet 2, exercises 2 and 3. The Function reference, Week 2, Files. The opening of py4e chapter 7, or Automate the Boring Stuff chapter 10, which writes `with open(...)`: fine, though our course style is plain `open` and `close`.

4 · Starting up: program arguments

Slides, page 23.

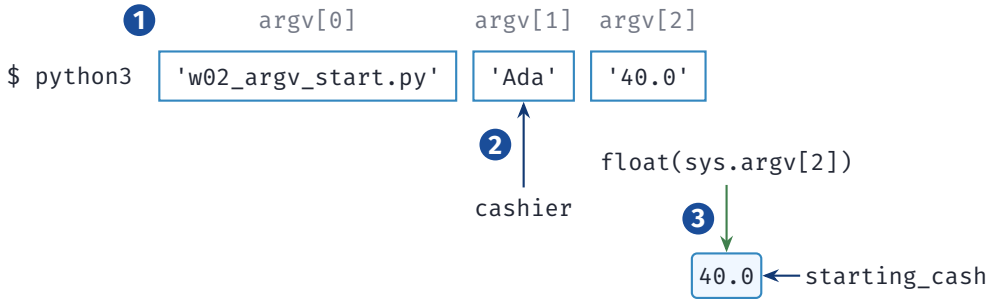
One more way in: the words you type after the program's name when you launch it. Import `sys`, and `sys.argv` holds them:

```
# w02_argv_start.py (on Brightspace: the road not taken, kept runnable)
import sys
cashier = sys.argv[1]                  # argv[0] is the program's own name
starting_cash = float(sys.argv[2])     # arguments arrive as strings -> convert
print(f"Register open. {cashier} starts the day with {starting_cash:.2f} in the drawer.")
```

```
$ python3 w02_argv_start.py Ada 40.0
Register open. Ada starts the day with 40.00 in the drawer.
```

Picture the command as a row of seats, numbered from 0 again. Seat 0 always holds the program's own name, `w02_argv_start.py`; your arguments sit in seats 1, 2 and on. They all arrive as strings, so convert them just like `input`. `sys.argv` itself is a list, a row of values in square brackets, and week 4 makes lists ours. Two import styles: `from sys import argv` gives you a bare `argv`; `import sys` gives you `sys.argv`, which I prefer, because you can always see where a name came from.

❶ Everything after `python3` takes a seat: `'w02_argv_start.py'` in seat 0, `'Ada'` in seat 1, `'40.0'` in seat 2, all text. ❷ `cashier = sys.argv[1]` puts the name `cashier` on seat 1's text. ❸ `float(sys.argv[2])` reads seat 2's text and builds a new value, the number 40.0, and `starting_cash` points at it. Leave out the cash and seat 2 does not exist, which is the `IndexError` below.



Leave out the second argument and the program stops on the line that asks for it:

```
$ python3 w02_argv_start.py Ada
IndexError: list index out of range
```

Python checks the grammar of the whole file first, as week 1 showed, then runs it line by line, and an error like this one only shows when its line runs. That cuts both ways: a mistake on a line your test run never reaches stays hidden, so a program that ran once is not yet a program that works.

What are arguments actually for? Picture a program that reads a file whose name is written into the code, `revenue.txt`. To run it on another file, you edit the code; on a third file, you edit it again. Read the name from `sys.argv[1]` instead, and the same program runs on any file without a single edit: you change the command, not the code. And a terminal tip: the up arrow brings back your last command, so you can rerun with new arguments without retyping everything. On Windows, type `python` where these notes say `python3`.

> **Good practice.** A program that takes arguments should fail with a usage line, not a traceback. Check `len(sys.argv)` first, and stop with `sys.exit("usage: python3 w02_argv_start.py NAME CASH")`, a call the slides do not cover: the message goes to the error channel, and the program ends with a status that tells the terminal it failed. Whoever runs it wrong learns how to run it right.

Common mistakes. A missing argument, above. Forgetting to convert: `sys.argv[2] * 2` repeats the text, it does not double the money. And counting from 1: your first argument is `sys.argv[1]`, because seat 0 is taken.

Check yourself.

- **4.1** What happens with `python3 w02_argv_start.py 40.0 Ada`, the two arguments swapped, and on which line?
- **4.2** For `python3 report.py march 12`, what are `sys.argv[0]`, `sys.argv[1]` and `sys.argv[2]`, and what type is each?

See also. Sheet 2, exercise 1. The Function reference, Week 2: the `sys.argv` row. Automate the Boring Stuff chapter 12, the one place either book covers command-line arguments.

5 · Comparing: boolean logic

Slides, pages 25 to 30.

Comparing data

Poll. The register now listens and remembers. What yes-or-no questions should it answer about an order? Is it a big order? Is the shop open? Has the customer paid? Each answer is a `bool`, week 1's `True` or `False`, and this section earns them with comparisons instead of typing them. Acting on an answer waits for week 3's `if`.

Page 25. A comparison asks a question and hands back an answer you can store:

```
>>> revenue = 17.0
>>> target = 150.0
>>> hit_target = revenue > target     # the answer is a VALUE...
>>> print(hit_target, type(hit_target))
False <class 'bool'>
```

Seventeen is not more than a hundred and fifty, so the answer is `False`, week 1's `bool`, now earned by a comparison rather than typed. The operators line up with mathematics: `<`, `<=`, `>`, `>=`, `==` and `!=`. Strings compare too, letter for letter, capitals included:

```
>>> print("espresso" == "latte", "print" == "Print")
False False
```

You can also ask whether one string is "less than" another, and it rarely means what you want, so for text stick to `==` and `!=`.

The classic: == compares, = assigns

Page 26. In mathematics one equals sign means equal. In Python one equals sign already means assignment, so equality takes two. Write one by accident and Python does not complain:

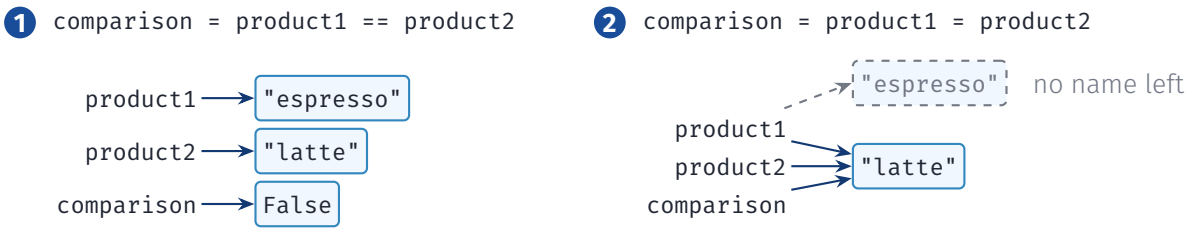
```
>>> product1 = "espresso"
>>> product2 = "latte"
>>> comparison = product1 == product2
>>> print(comparison)
False
>>> comparison = product1 = product2
>>> print(comparison)
latte
```

`comparison = product1 = product2` is an assignment chain: `comparison` and `product1` now both point at `"latte"`, and `"espresso"` is gone. No question was asked, no error was raised, and the program runs on, wrong. Your tell: a comparison that prints a value instead of `True` or `False`. On a line of its own, Python catches the slip only when the left side is a value it cannot assign to:

```
>>> "latte" = "latte"
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?
```

With names, as above, you are on your own. Next week a single `=` inside an `if` is a `SyntaxError` whatever the left side, so that one Python does catch.

1 With `==`, Python asks a question: the two names keep their strings, and `comparison` gets the answer, `False`. 2 With `=`, nothing is asked: the chain points `comparison` and `product1` at `"latte"`, and `"espresso"`, with no name left, is gone.



Combining questions: and, or, not

Page 27. Real questions have more than one part. Did we serve ten or more customers today, and was the revenue above 150?

```
>>> customers = 12
>>> revenue = 145.0
>>> print(customers >= 10 and revenue > 150.0)
False
```

Twelve clears the first part, 145 does not clear the second, and `and` needs both. The three words read like English: `and` needs every part true, `or` needs at least one, and `not` flips a single answer, so `not (interruptions > 5)` is true when there were five interruptions or fewer.

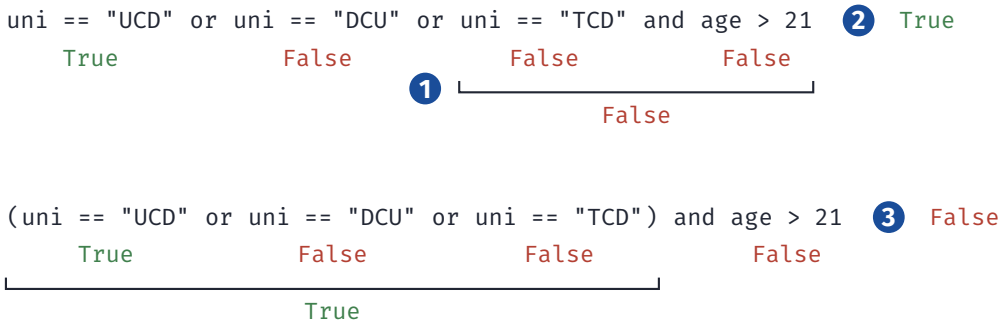
The precedence trap

Page 28. This one is sneaky, because it gives a plausible wrong answer:

```
>>> uni = "UCD"
>>> age = 18
>>> print(uni == "UCD" or uni == "DCU" or uni == "TCD" and age > 21)
True
>>> print((uni == "UCD" or uni == "DCU" or uni == "TCD") and age > 21)
False
```

How did an eighteen-year-old pass an over-21 check? Like `*` before `+` in arithmetic, the three words have an order: `not` first, then `and`, then `or`. So the first line pairs the age test with TCD only: UCD, or DCU, or the pair TCD-and-over-21. Write each part's answer under it and the line reads `True or False or (False and False)`: the part in parentheses is `False`, and one `True` is enough for `or`. With parentheses around the three universities, the age applies to all of them, and the same student correctly gets `False`. Do not memorize the order under exam pressure: use parentheses. They cost nothing and they say what you mean.

Here is each part's answer, written under it. ❶ In the first line `and` goes before `or`, so it takes only its two neighbors, the TCD test and the age test: `False and False` is `False`. ❷ What is left is `True or False or False`, and one `True` is enough. ❸ In the second line the parentheses make the three universities one question, `True`, and the age test applies to all of it: `True and False` is `False`.



Truth tables

Page 29.

`x`	`y`	`x and y`	`x or y`
False	False	False	False
False	True	False	True
True	False	False	True
True	True	**True**	True

`and` is true only on the last row; `or` is false only on the first; `not` turns `True` into `False` and back. Learn these by heart: the exam is closed book, and in week 9 the decisions they feed are what we count. The more you practice, the less you will need to look.

The register answers questions, and hits a wall

Page 30. Into the register the booleans go:

```
big_order = total > 10.0
busy_start = espresso_qty + latte_qty + muffin_qty >= 5
print(f"Big order (over 10.00)?  {big_order}")
print(f"Busy start (5+ items)?   {busy_start}")
```

For the seventeen-euro, six-item order, both answers are `True`. Twenty-two muffins and one latte, 56.00, give `True` twice again. These are flags, like last week's "is the shop open?", and you can imagine what a café would do with them: a discount on a big order, a tip for the staff on a busy start. That is exactly the wall, and it is deliberate: the register can compute a decision and cannot act on it. "If it is a big order, apply a discount": we can write the first half of that sentence in Python now, and not the second. The missing word is `if`, and it is next week.

Common mistakes. One `=` where you meant `==`. An `and` mixed with an `or` without parentheses. And comparing text that differs only in capitals, which is never equal.

Check yourself.

- **5.1** When is `not (stock < 3)` true? Write the same test without `not`.
- **5.2** Write `a and b or not c` with the parentheses Python silently uses.
- **5.3** `size = "large"` and `wanted = "small"`. A line meant to ask whether they match reads `match = size = wanted`. What does `print(match, size)` show, and what should the line say?

See also. Sheet 2, exercise 1 (d). The Function reference, Week 2: the comparison and `and` `or` `not` rows.

6 · Functions

Slides, pages 31 to 36.

Same logic, different data

Page 31. You have called functions since the first minute: `print`, `type`, `len`, `input`, `open`, `int`. Each takes something in, does a job, and often hands something back, and `help(print)` shows you `print`'s own parameters. Now we write our own:

```
def line_total(price, quantity):
    return price * quantity
```

The word `def`, a name, the parameters in parentheses, a colon, then an indented body; `return` hands a value back to whoever called. Why bother? The same question, asked many times with different data, should be written once and called forever.

Poll. Type the header of a function `price` that takes one argument `qty`. It is `def price(qty):`. Read it the way you read a call to `print`: the name, then the parameter in parentheses. Then add the two pieces a call never has, the word `def` in front and the colon at the end. Leave out the colon and Python stops before running anything, with `SyntaxError: expected ':'`.

Indentation is the syntax

Page 32. In many languages indentation is decoration. In Python it is grammar: an indented line belongs to the `def` above it, the line that ends in a colon, and the moment the indentation stops, the body is over.

```
def big_order_check(total):
    threshold = 10.0            # <- 4 spaces: "I belong to this def"
    return total > threshold
```

Python does not fix the amount, as long as every line of a body uses the same one; the standard, and this course's rule, is four spaces. Agree on it in your project group, and never mix tabs and spaces: they look identical on screen and break in ways that waste an evening. A line that steps back to a depth Python never saw stops the program before it starts:

```
IndentationError: unindent does not match any outer indentation level
```

In the shell, the body shows a `...` prompt, and a blank line finishes the function. From Python 3.13 the shell may also indent for you: after a line that ends in a colon, the cursor starts the next line four spaces in, and the lines after it keep that depth. So look at the cursor before you type a body line. Already four spaces in: type the line without the spaces shown here, press Backspace once to step back a level, as before an `else` or an `except`, and press Enter on the empty line to finish. Spaces typed on top of the shell's stop the second body line with `IndentationError: unexpected indent`. At the left edge, as in Python 3.12, which indents nothing: type the spaces yourself.

A function you define does nothing until you call it. Write a perfectly valid function, run the file, and nothing happens: nothing called it. Call it without the argument it needs and Python says so:

```
>>> def price(qty):
...     return qty * 2
...
>>> price()
TypeError: price() missing 1 required positional argument: 'qty'
```

A parameter can also take a default value, `def price(qty=0):`, which a call with no argument then uses.

Good practice. When a call passes several values of one type, name them: `receipt_line(product="latte", price=3.20, quantity=3)` reads without a trip back to the `def`, and two swapped numbers can no longer land in the wrong parameters. It is the same `name=value` you give `print` as `sep`. And put parameters with defaults last: Python refuses `def price(qty=0, size):` before the program starts.

Printing is not returning

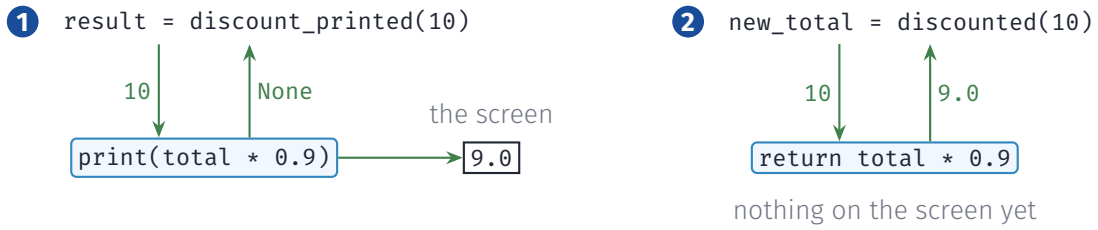
A function that prints shows a value and keeps nothing; a function that returns hands the value back, to be stored, printed or passed on. Here is a discount written the first way:

```
>>> def discount_printed(total):
...     print(total * 0.9)
...
>>> result = discount_printed(10)
9.0
>>> print(result)
None
```

It shows 9.0 and hands back nothing, which Python spells `None`: the 9.0 is on the screen and nowhere else. So we made it the second kind:

```
>>> def discounted(total):
...     return total * 0.9
...
>>> new_total = discounted(10)
>>> print(new_total)
9.0
>>> def is_good_discount(total):
...     return total * 0.1 > 5
...
>>> print(f"Good price? {is_good_discount(10)}")
Good price? False
>>> print(f"Good price? {is_good_discount(1000)}")
Good price? True
```

Drawn side by side, the difference is where the 9.0 goes. (1) `discount_printed` sends 9.0 to the screen, and what comes back to `result` is `None`. (2) `discounted` sends 9.0 back to the caller, and `new_total` holds it, to print, to store or to pass on. Printing shows a value to a person; returning hands it to the program.



A function can return a comparison, which makes it a yes-or-no question with a name. And functions feed functions: in `int(input(...))`, `input`'s answer is eaten by `int`, whose answer you can hand to `discounted`. That is the `x`, `f`, `y` chain from the start of the lecture, in code.

Zero or more in, zero or more out

Page 33.

```
def hello():                          # no inputs, no return (it just DOES something)
    print("Hello!")

def cube(nbr):                        # one in, one out
    return nbr * nbr * nbr

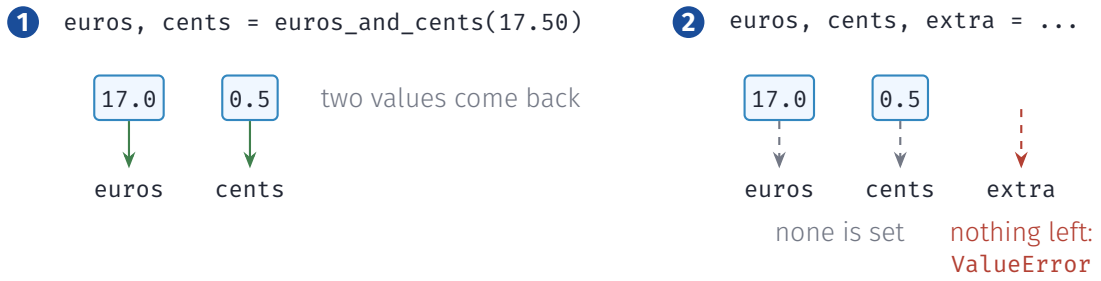
def euros_and_cents(amount):          # one in, TWO out
    whole = amount // 1.0
    return whole, amount - whole
```

`//` is week 1's floor division: divide, and keep the whole part, so `17.50 // 1.0` is `17.0`. `euros_and_cents` returns two values, separated by a comma, and the caller can catch both at once with two names on the left. Catch them in one name and you get both together, in parentheses, a shape week 4 names:

```
>>> print(euros_and_cents(17.50))
(17.0, 0.5)
>>> euros, cents = euros_and_cents(17.50)   # euros = 17.0, cents = 0.5
>>> euros, cents, extra = euros_and_cents(17.50)
ValueError: not enough values to unpack (expected 3, got 2)
```

The names on the left have to match the values coming back, one for one. Here `cents` is the fraction of a euro, 0.5; the café's receipt multiplies it by 100 to print it as whole cents.

1 The function hands back two values, 17.0 and 0.5, and two names on the left take one each, in order. 2 Three names for two values leave the third with nothing to take, so Python sets none of the three and stops with the `ValueError` above.



Not every amount splits as cleanly as 17.50:

```
>>> print(euros_and_cents(2.30))
(2.0, 0.2999999999999998)
```

Week 1's float wall again: 2.30 is stored a hair below itself. The receipt still prints 30 cents, because it formats `cents * 100` with `:.0f`, which rounds; `int(cents * 100)` would cut 29.999999999999982 down to 29.

The receipt, refactored

Page 34. The heart of the section. Week 1's receipt was copy-paste: near-identical lines edited by hand. Now it is one rule, reused:

```
def receipt_line(product, price, quantity):
    """One aligned receipt row -- the format specs do the layout work."""
    return f"{product:<12} x {quantity:>2} {line_total(price, quantity):>8.2f}"
```

```
print(f"{' The Daily Grind ':=^34}")
print(receipt_line("espresso", espresso_price, espresso_qty))
print(receipt_line("latte", latte_price, latte_qty))
print(receipt_line("muffin", muffin_price, muffin_qty))
print("=" * 34)
```

Read the row's spec piece by piece: the product left-aligned in 12 characters, then `x`, then the quantity right-aligned in 2, a space, and the line total right-aligned in 8 with two decimals. `receipt_line` calls `line_total` for the arithmetic: functions build on functions. With the rule written once, a new product costs one line, not four, and when the order gets a memory in week 4, this is the function that prints every row of it. Run the café again with the same order and the receipt is identical: a refactor changes the structure and not the behavior. Keep that sentence for week 7.

Good practice. Give every function a one-line docstring, as `receipt_line` does, and think about type hints, which the slides do not cover: `def line_total(price: float, quantity: int) -> float:` says what goes in and what comes out. Python ignores hints when it runs; your editor reads them, and so does the next programmer. And keep each function to one job, named for that job.

Follow a call by hand

Page 35. "State the output of this program" in the exam is exactly this walk. Predict the printed row before you read the table:

```
def line_total(price, quantity):
    return price * quantity

def receipt_line(product, price, quantity):
    return f"{product:<12} x {quantity:>2} {line_total(price, quantity):>8.2f}"

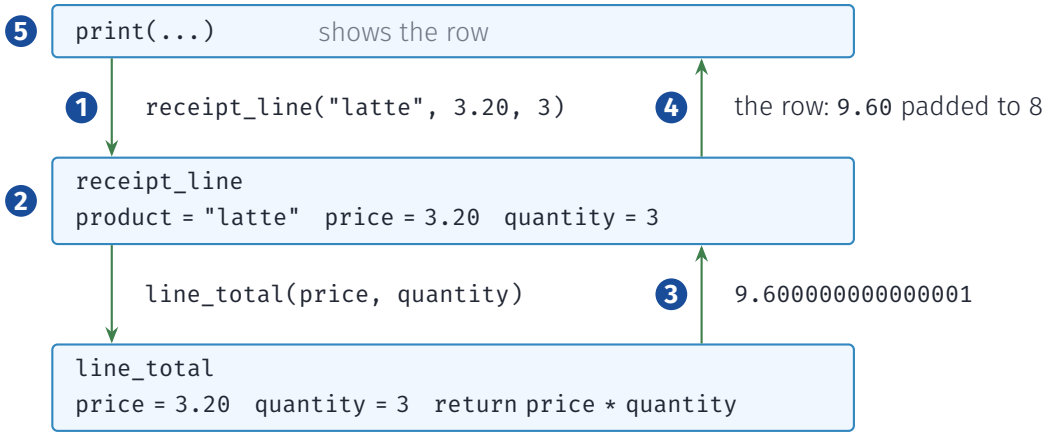
print(receipt_line("latte", 3.20, 3))
```

Step	Where	What happens
1	`print(...)`	needs `receipt_line("latte", 3.20, 3)` first
2	`receipt_line`	`product="latte"`, `price=3.20`, `quantity=3`
3	`line_total`	`price=3.20`, `quantity=3`: returns `9.6...`
4	back in the f-string	`9.6...` formatted `:>8.2f`: `9.60` padded to 8 characters
5	back in `print`	the full row appears

The row printed:

```
latte        x  3     9.60
```

Read the picture down the left, then up the right; its steps are the table's. ❶ `print` needs its argument first, so `receipt_line` is called, and ❷ its parameters take the three values in order. Inside its f-string, `line_total(price, quantity)` is called in turn, and ❸ returns the product, 9.600000000000001. ❹ Only then does the f-string go on: it formats the number, 9.60 padded to 8 characters, and `receipt_line` hands the finished row back. ❺ `print` shows it. A call waits for every call inside it, so the innermost one finishes first.



Defining runs nothing: Python reads each `def`, remembers the function, and moves on. The bodies run only when the last line calls them, from the innermost call outward, the way parentheses work in arithmetic. So what matters is that every function exists before the line that calls it runs. The order of the `def`s among themselves does not: put `line_total` below `receipt_line` and it still works, because `receipt_line`'s body only runs at the call, and by then both exist.

```
>>> def receipt_line(product, price, quantity):
...     return f"{product:<12} x {quantity:>2} {line_total(price, quantity):>8.2f}"
...
>>> def line_total(price, quantity):
...     return price * quantity
...
>>> print(receipt_line("latte", 3.20, 3))
latte        x  3     9.60
```

What fails is a call that runs before its function exists: `print(square(4))` on a line above `def square` raises `NameError: name 'square' is not defined`.

Python reads a file top to bottom. (1) Each `def` makes its function and remembers its name; the body waits. (2) The call on the last line runs both bodies, since both exist by then. (3) Put a call above its `def` and Python reaches the call first: the name does not exist yet, and the run stops with `NameError` before the `def` below it ever runs.

every call after its `def`: it works

```
def receipt_line(...): ...
def line_total(...): ...
print(receipt_line("latte", 3.20, 3))
```

(1) both remembered; the bodies wait

(2) both exist: the bodies run

a call before its `def`: it stops

```
print(square(4))
def square(...): ...
```

(3) `NameError`: no `square` yet

never reached

Your turn

Page 36.

Poll. The drill ends with `print(pi)`, on a line outside the function that set `pi` to 3.14. What does that line show? Predict it before you read the run below; the reason is the door to the next section.

```
>>> def greet(name): print(f"Hello, dear {name}!")
...
>>> def leave(): print("I'm leaving now, goodbye!")
...
>>> greet(input("What's your name? ")); leave()
What's your name? Ada
Hello, dear Ada!
I'm leaving now, goodbye!
>>> def area_of_ellipse(semi_major, semi_minor):
...     pi = 3.14
...     return pi * semi_major * semi_minor
...
>>> print(area_of_ellipse(1.0, 1.0))
3.14
>>> print(pi)
NameError: name 'pi' is not defined
```

`input` feeds `greet` directly: the innermost call again. And the last line is the trap, sprung on purpose: `pi` was born inside the function and died when the function returned. Out here it never existed. That `NameError` is the door to the next section.

Common mistakes. Defining a function and never calling it. Calling it with too few arguments. Indentation that does not line up. Calling a function on a line that runs before its `def`. And catching two returned values in three names, or three in two.

Check yourself.

- **6.1** Trace this program and state its output.

```
def with_tax(price):
    return price * 1.10

def label(name, price):
    return f"{name}: {with_tax(price):.2f}"

print(label("scone", 3.00))
```

- **6.2** In this section's reordered program, `receipt_line` defined first, move the `print` line between the two `def`s. What happens, and why?

See also. Sheet 2, exercise 5. The Function reference, Week 2: the `def` and `return` rows. py4e chapter 4, or Automate the Boring Stuff chapter 4.

7 · Scope

Slides, pages 37 and 38.

Where a name lives

Page 37. That `NameError` has a rule behind it: a name born inside a function lives only inside that function.

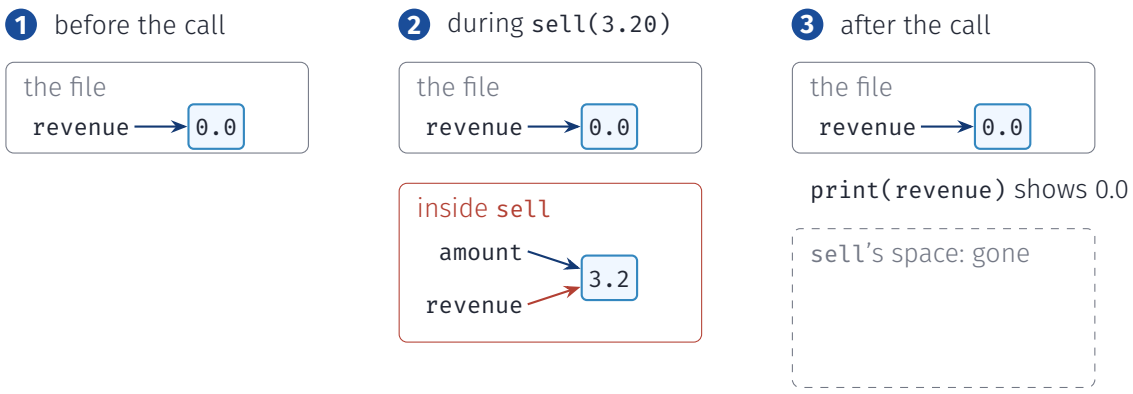
```
def hello():
    msg = "Hello, world!"      # msg is LOCAL: born in the function, dies with it
    print(msg)

hello()
print(msg)                     # NameError: name 'msg' is not defined
```

That is **local scope**. The other direction is friendlier: a **global**, a name created at the top level of the file before the call runs, can be read inside a function. Set it only after the call runs and the call raises `NameError`, because the name does not exist yet when the body looks for it; week 5 turns that into a rule for laying out a whole file. Two complications. A local with the same name as a global wins inside the function: it **shadows** the global. And assigning to a name inside a function makes a new local; it does not change the global, even when the names match:

```
>>> revenue = 0.0
>>> def sell(amount):
...     revenue = amount
...
>>> sell(3.20)
>>> print(revenue)
0.0
```

1 Before the call, the file has one `revenue`, 0.0. 2 The call opens a space of its own: `amount` holds 3.2, and `revenue = amount` makes a second `revenue` there, a local, also on 3.2. 3 When the call returns, its space and both its names are gone, and the file's `revenue` never moved: `print` shows 0.0.

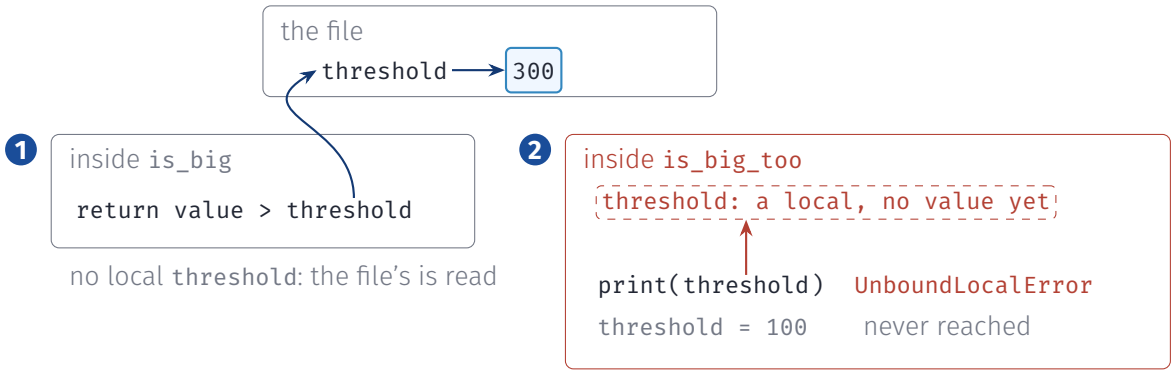


The rule has a sharp edge. Python decides once, for the whole function, whether a name is local: if the function assigns to it anywhere, it is local everywhere in that function, even on the lines above the assignment.

```
>>> threshold = 300
>>> def is_big(value):
...     return value > threshold        # only reads it: the global is used
...
>>> print(is_big(200))
False
>>> def is_big_too(value):
...     print(threshold)                # assigned below, so local: not set yet
...     threshold = 100
...     return value > threshold
...
>>> print(is_big_too(200))
UnboundLocalError: cannot access local variable 'threshold' where it is not associated with a value
```

`is_big` reads the global and works. `is_big_too` assigns `threshold` on its second line, so `threshold` is local from its first line, and on that first line it has no value yet. Same name, two different variables. That isolation is a good thing: a name a function assigns stays in the function, so no function can change a global by accident. The `global` keyword can force the issue; it is ugly, and my advice is plain: avoid globals unless you truly have no choice.

Before a function runs, Python looks for assignments in it. 1 `is_big` assigns nothing, so `threshold` is the file's, 300. 2 `is_big_too` assigns `threshold` on its second line, so `threshold` is a local for the whole body, and on the first line that local has no value yet: `UnboundLocalError`, and the file's 300 is never looked at.



Design and why: `revenue` is not global

Page 38. A design slide, no syntax. The lazy design makes `revenue` a global and lets every function add to it. It works, right up until you cannot tell which function changed the money, or you try to test one piece on its own and it depends on hidden state. So the café takes a rule instead: **a function receives what it needs through its parameters and returns what it produces.** `total = line_total(price, qty)`: the data comes in the front door, the answer goes out the front door, and nothing reaches around the side. That is the week-2 seed of the biggest design idea in the course, one clear owner for every piece of state, which week 7 names properly. Your project lives or dies on this discipline, so I am planting it early on purpose.

> **Good practice.** Avoid `global` altogether. A function that reads only its parameters and hands back a result can be understood, tested and reused on its own; a function that reaches for globals cannot. When a value must be shared, make it a constant in capitals at the top of the file, and never assign to it again.

> **In the project.** The Project rules mark structure, modules and single-purpose functions, and they mark the split this section starts: the agent's thinking kept apart from the desk that owns the money. Week 7 builds that split with classes.

Common mistakes. Using a function's local after the function returned. Expecting an assignment inside a function to change the global of the same name. And reading a name in a function that also assigns it further down.

Check yourself.

- **7.1** What prints? And what prints if the body of `check` starts with the line `limit = 20`?

```
limit = 10

def check(x):
    return x > limit

print(check(12))
```

- **7.2** What happens when this file runs?

```
def show():
    print(opening_cash)

show()
opening_cash = 40.0
```

See also. Sheet 2, exercise 5 (b). Automate the Boring Stuff chapter 4, the best free coverage of scope.

8 · Modules

Slides, pages 39 and 40.

Don't reinvent the wheel

Page 39. Python ships with batteries included: collections of ready-made code called modules. `sys` holds our `argv`; `math` holds proper mathematics; `os` talks to the operating system, `random` makes random numbers and `datetime` handles dates. `print` and `input` are built in, available without an import. For the rest, you import the module and reach into it:

```
>>> import math
>>> print(math.sqrt(2))          # module.member: you can SEE where names come from
1.4142135623730951
```

Week 1's `**` does powers; the square root lives in `math`. The same two styles as with `sys`: `import math` and then `math.sqrt`, which I prefer because the name tells you where it came from, or `from math import sqrt` and a bare `sqrt`. Thousands more modules are a download away, and you can write your own.

Our first two-file program

Page 40. `register_sayings.py` holds a `welcome` function and a `goodbye` string, nothing else:

```
# register_sayings.py -- the cafe's first MODULE
def welcome(name):
    return f"Welcome to The Daily Grind, {name}!"

goodbye = "Thank you, come again!"
```

One line in `cafe_w2.py`, `import register_sayings`, and the café can call `register_sayings.welcome` and read `register_sayings.goodbye`, with the same module-dot-member spelling as `math.sqrt`:

```
>>> import register_sayings
>>> print(register_sayings.welcome("Ada"))
Welcome to The Daily Grind, Ada!
>>> print(register_sayings.goodbye)
Thank you, come again!
```

"Welcome to The Daily Grind" is nowhere in `cafe_w2.py`. I never wrote it there: it lives in the other file, and the import brought it in. Two rules to take away. A module is a Python file, and you import it by its name without `.py`. And a module runs top to bottom the moment it is imported, so it should hold declarations, functions and variables, and never surprise you with a `print` firing on import. The café is officially multi-file, like every real program.

> **Good practice.** Make every file safe to import: keep the code that runs the program in a `main()` function, called from one guarded line at the bottom, as the café's files do from week 3 on. Week 5 explains that line and makes it the course's file layout, and your project follows it.

Common mistakes. Writing the extension, `import register_sayings.py`, or keeping the two files in different folders. Python's answers, in that order:

```
ModuleNotFoundError: No module named 'register_sayings.py'; 'register_sayings' is not a package
ModuleNotFoundError: No module named 'register_sayings'
```

And calling `register_sayings.welcome` without the import line, which is a `NameError`.

Check yourself.

- **8.1** Why should a module hold functions and variables, and no `print` at the top level?
- **8.2** Rewrite `import register_sayings` in the other import style, so the café can write a bare `welcome`. What does the greeting line become?

See also. Sheet 2, exercise 5. The Function reference, Week 2: the `import NAME` and `math.sqrt` rows.

Where the café is now

Slides, pages 41 to 44.

Stand back and look at what the café does now. It takes a real order from the keyboard, prints an aligned receipt where the money is finally honest, answers yes-or-no questions about the order, and saves revenue that survives the night. That is a real little program, in two files, and you built it. Now the walls we hit on purpose, because naming them is half of next week's motivation:

- the order form is hard-coded, one question per product, with no "until done": that is a **loop**, week 3;
- `big_order` comes back `True` and the register cannot act on it: that is **`if`**, week 3;
- reading a file that might not exist, or a quantity that is not a number, crashes: that is **exceptions**, week 5.

Next week the register gets a menu loop and its first real decisions, and we start drawing the program before we write it.

Read `cafe_w2.py`, all of it. Each week's café uses only the tools we have covered by then, so a line that puzzles you is a gap worth closing now, before the weeks that build on it. The practical sheets use the same tools on a different program, which makes the café a worked example for them.

What the exam can ask from this week, as page 42 lists it: state the output of format specs and of function calls (the trace table); spot the `input()`-returns-a-string `TypeError`, the `=` versus `==` bug and the `and`/`or` precedence trap; explain local versus global scope; complete a one-line function; pick the right file mode, `"w"` or `"a"`; unpack a function's two return values; import your own module.

The checklist on page 44 is your revision list. Anything you cannot tick, read its section here again, then do the matching exercise on sheet 2. Self-check 2 asked the kinds of questions the exam asks, and every week builds on the one before, so a gap here costs you again in every week after it.

Practice it closed-book

The exam is closed book, so practice the way it runs: from memory, on paper, then check.

- Cover the outputs of sections 1, 5 and 6 and the pictures beside them: the format specs, the `.format` calls, the `=` chain, the precedence trap and the unpacking of `euros_and_cents`. Write each one character for character, spaces included; then look.
- Write `line_total`, `receipt_line` and `euros_and_cents` from memory and compare them with pages 33 and 34; then say in one sentence why the café opens `revenue.txt` with `"w"` and `receipts.txt` with `"a"`.
- Trace `receipt_line("muffin", 2.40, 4)` in a table like page 35's, write the row it prints, then run it to check.
- Write the `and`/`or` table and the `not` table of page 29 from memory.
- Cover the scope demos of section 7 and their pictures, and predict each one, the errors included.

- In a few days, answer the check-yourself questions again without these notes, then take the week's self-check by hand, with Python closed.

Answers to check yourself

- **1.1** `espresso 2.50`: the f-string was built while `price` was 2.50, and rebinding `price` afterward does not reach back into it.
- **1.2** `*cocoa**`: a field of 8 holds the five letters and three stars of fill, and since three stars do not split evenly, the extra one goes on the right.
- **1.3** `tac-tic-tac`: `{1}` is the second value, `{0}` the first, and a number can be used twice.
- **2.1** `44`: `qty` is the text `"4"`, and text times 2 repeats it. `int(qty) * 2`, or `int(input(...))` on the way in, gives `8`.
- **2.2** `input` runs first, the innermost call, and hands back `'Ada'`; `len` counts its three characters; `print` runs last and shows `3`.
- **3.1** Two lines, `2.00` then `0.75`: `"w"` emptied the file before writing, so `1.50` is gone, and `"a"` added to the end.
- **3.2** `menu` is a file object, a pointer to the file plus its bookmark, not the text. `menu.read()` gives the contents, or `menu.readline()` one line at a time.
- **3.3** `' single shot 2.50\n'`. The first nine characters are the word espresso and its comma, so the bookmark sits just after the comma, and the line comes back from there, its leading space included.
- **4.1** It stops with a `ValueError` on the line that converts `sys.argv[2]`, and the message reads `could not convert string to float: 'Ada'`. Seat 1 now holds `40.0`, which becomes the cashier's name without complaint; seat 2 holds `Ada`, which `float` cannot read.
- **4.2** `'report.py'`, `'march'` and `'12'`, and all three are strings, the `12` too.
- **5.1** When `stock` is 3 or more: `stock >= 3`.
- **5.2** `(a and b) or (not c)`: `not` first, then `and`, then `or`.
- **5.3** `small small`: one `=` made an assignment chain, and `size` lost its value. It should read `match = size == wanted`, which gives `False`.
- **6.1** `scone: 3.30`: `print` needs `label` first, `label` calls `with_tax`, which returns 3.3 with a tiny float tail, and `:.2f` shows it as `3.30`.
- **6.2** It stops with this error, raised inside `receipt_line`:

  ```
  NameError: name 'line_total' is not defined
  ```

  That function exists, since its `def` came first, so the call runs its body; the body calls `line_total`, whose `def` has not run yet.
- **7.1** `True`: `check` only reads `limit`, so it uses the global 10. With `limit = 20` first in the body, `limit` is local, 12 is not above 20, and it prints `False`.
- **7.2** `NameError: name 'opening_cash' is not defined`, raised inside `show`: when the call runs, the global does not exist yet.
- **8.1** A module runs top to bottom the moment it is imported, so a top-level `print` would fire in every program that imports it, whether that program wants it or not.
- **8.2** `from register_sayings import welcome`, then `print(welcome(cashier))`.